

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY,
HYDERABAD

Statistical Methods in Artificial Intelligence — Assignment 3

Hindi Product Review Sentiment Analyser

Variant T8.1 · Theme T8: Indic Review & Sentiment Analysis

Model:	Fine-tuned <code>ai4bharat/indic-bert</code> (ALBERT-based)
Baseline:	<i>LLaMA 3.3 70B</i> via GROQ (Zero-shot)
Dataset:	AI4Bharat IndicSentiment — Hindi split
Classes:	Binary: Positive / Negative
Test Acc.	IndicBERT: 82.76% GROQ: 100%
Tier:	Tier 1 (1–3 students)

Authors

Rajai Tarun Kanaiyalal - 2025201034 - tarun.rajai@students.iiit.ac.in

Aryan - 2025201017 - aryan.aryan@students.iiit.ac.in

Parimal Mate - 2025201019 - parimal.mate@students.iiit.ac.in

Academic Year 2025–26

<https://github.com/RajaiTarun/Hindi-product-review-sentiment>

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Contributions	3
2	Data	4
2.1	Dataset Overview: AI4Bharat IndicSentiment (Hindi)	4
2.2	Exploratory Data Analysis	4
2.2.1	Label Distribution	4
2.2.2	Review Length Statistics	4
2.2.3	Sample Reviews	5
2.3	Dataset Columns	5
3	Method	6
3.1	Approach A: Supervised Fine-Tuning of IndicBERT	6
3.1.1	Model Architecture: <code>ai4bharat/indic-bert</code>	6
3.1.2	Tokenisation	7
3.1.3	Training Configuration	7
3.1.4	Epoch Selection: The Case for 7 Epochs	7
3.1.5	Regularisation Strategy	8
3.1.6	PyTorch Dataset and DataLoader	8
3.2	Approach B: Zero-Shot LLM Baseline via GROQ	8
3.2.1	Motivation for a Zero-Shot Comparison	8
3.2.2	Model Selection and Dynamic Switching	9
3.2.3	Prompt Engineering	9
4	Results	9
4.1	Primary Metrics Overview	9
4.2	IndicBERT: Detailed Evaluation	9
4.2.1	Classification Report	10
4.2.2	Confusion Matrix Analysis	10
4.2.3	Training Curves and Convergence	10
4.3	Zero-Shot (GROQ LLaMA 3.3 70B): Detailed Evaluation	11
4.3.1	Perfect Score Analysis	11
4.3.2	Confidence Distribution	11
5	Ablation Study	12
5.1	Epoch Ablation	12
5.2	Supervised Fine-Tuning vs Zero-Shot	12
6	Web Application	12
6.1	Confidence Score Asymmetry: Positive vs. Negative	13
6.2	Top Themes Detection: Methodology	13
6.2.1	Keyword Dictionary Design	13
6.2.2	Counting Mechanism	14
6.2.3	Advantages of the Approach	14

7	Limitations	14
8	Demo Video	15
9	LinkedIn Pitch Slide	15
10	References	15
	Acknowledgements	16
11	Application Screenshots	16

1 Introduction

India hosts one of the world’s fastest-growing e-commerce ecosystems, with platforms such as Flipkart, Amazon India, and Meesho collectively handling hundreds of millions of product transactions annually. A significant fraction of customer feedback on these platforms is written in **Hindi**—the mother tongue of approximately 600 million people—yet the overwhelming majority of commercially available sentiment analysis tools are designed exclusively for English. This linguistic disparity leaves small-business owners, D2C brands, and marketers unable to extract structured, actionable insights from their most voluminous stream of feedback.

This project bridges that gap by constructing a **Hindi Product Review Sentiment Analyser**: an end-to-end machine-learning pipeline that reads a Hindi product review written in Devanagari script and classifies it as either *Positive* or *Negative*, while additionally reporting a calibrated confidence score.

Although the original AI4Bharat IndicSentiment benchmark supports a three-class schema (Positive, Negative, Neutral), the actual Hindi (hi) JSONL split fetched from HuggingFace at experiment time contained *only two classes*—Positive and Negative. No neutral samples were observed in the downloaded data. Accordingly, **all modelling, evaluation, and deployment are conducted under the binary classification assumption**. Sample CSV files generated for the Streamlit demo and bulk-upload tests also adhere to this two-class schema.

1.1 Motivation

Sentiment analysis for Indic languages is an active research frontier. While transformer-based models have revolutionised English NLP, the Indic NLP community has historically suffered from: (i) insufficient labelled corpora, (ii) morphological complexity that defeats whitespace tokenisation, and (iii) the Devanagari script’s distinct Unicode plane, which is invisible to many byte-pair-encoding vocabularies trained primarily on Latin-script text. The release of `ai4bharat/indic-bert` by AI4Bharat—an ALBERT-based model pre-trained on 12 Indic languages using a shared SentencePiece vocabulary—makes supervised fine-tuning tractable even on commodity GPUs, opening the door to portfolio-grade demonstrations of Indic NLP.

1.2 Contributions

- **Supervised fine-tuning** of `ai4bharat/indic-bert` for binary Hindi sentiment classification, achieving 82.76% test accuracy and a macro-F1 of 0.8274 over 7 epochs, with AdamW optimisation and linear learning rate decay.
- **Zero-shot LLM baseline** using Meta’s *LLaMA 3.3 70B* served via GROQ’s LPU inference API, achieving 100% accuracy on the 116-sample test set through structured JSON prompting.
- **Ablation** comparing fine-tuned supervised learning against zero-shot prompting, with quantitative metrics and discussion of the practical trade-offs.

- A **Streamlit web application** supporting single-review real-time inference, bulk CSV upload with downloadable results, and an interactive analytics dashboard.

2 Data

2.1 Dataset Overview: AI4Bharat IndicSentiment (Hindi)

The primary corpus is the [AI4Bharat IndicSentiment](#) benchmark, a multilingual sentiment dataset covering 13 Indic languages with parallel reviews sourced from Indian product and service review platforms. The Hindi configuration is referenced by the ISO code `hi`.

The dataset ships in JSONL format and was accessed directly via raw HuggingFace URLs (bypassing the now-deprecated custom loading script in `datasets` v4+), yielding two usable files:

File	Rows	Classes present	Source URL suffix
<code>validation/hi.json</code>	156	Positive, Negative	<code>.../validation/hi.json</code>
<code>test/hi.json</code>	1000	Positive, Negative	<code>.../test/hi.json</code>
Combined	1154	2	—

The combined pool of 1154 samples (after dropping rows with missing labels or reviews) was randomly stratified-split into a canonical 80/10/10 partition:

Split	Samples	Negative / Positive
Train	923	458 / 465
Validation	115	≈ 57 / 58
Test	116	58 / 58
Total	1154	—

The class balance is remarkably uniform: across all splits, Positive and Negative samples are within ± 1 of parity, with a near-perfect 50/50 ratio in the test set (58 each). This means standard accuracy is an appropriate primary metric and no class-weighting was required; nonetheless, we report macro-averaged F1 as the canonical metric to maintain comparability with future work on imbalanced extensions.

2.2 Exploratory Data Analysis

2.2.1 Label Distribution

As confirmed by exploratory analysis, the downloaded Hindi split contains exclusively binary labels—**Positive** and **Negative**. The label mapping assigned at runtime is:

$$\text{Negative} \mapsto 0, \quad \text{Positive} \mapsto 1.$$

2.2.2 Review Length Statistics

Review lengths were measured in both character count and whitespace-delimited word count:

Label	Character Length				Word Count			
	Mean	Std	Min	Max	Mean	Std	Min	Max
Negative	122.7	62.4	14	403	25.0	12.9	3	81
Positive	147.7	77.7	25	516	29.1	15.3	4	100

Positive reviews are consistently longer than negative ones (mean $\Delta_{\text{word}} \approx 4$), which is a common pattern in product review corpora: satisfied customers often elaborate on features they enjoyed, whereas negative reviews tend to be shorter and more pointed. This length disparity is an implicit feature that the transformer’s attention mechanism can exploit through position-aware representations.

A maximum sequence length of **128 tokens** was chosen for tokenisation. Given that the mean word count is ≈ 27 words and Hindi SentencePiece tokens are generally sub-word, the vast majority of reviews fit within this budget; very long reviews (>90 words) are truncated at the right-hand side, with negligible information loss in practice because sentential sentiment cues tend to cluster near the beginning and end of reviews.

2.2.3 Sample Reviews

Hindi: “निर्माण की क्वालिटी बुरी है क्योंकि कुछ पीस में छेद हैं, यह लंबे समय तक चलने वाला नहीं है।”

English: “Manufacturing quality is poor as some pieces come with a hole and it is not long-lasting.”

Hindi: “जरूर खरीदें!! सबसे अच्छी एक्टिविटी ओरिएंटेड बुक जो मैंने कभी देखी है। यह कलरफुल है...”

English: “MUST BUY!! The best activity-oriented book I have ever seen. It is colourful...”

These examples illustrate both the linguistic richness of Hindi text (compound verbs, mixed Devanagari-Roman loanwords like “क्वालिटी”) and the practical challenges for tokenisation.

2.3 Dataset Columns

The JSONL schema provides rich metadata beyond the review text and label:

Column	Description
GENERIC CATEGORIES	Broad product category
CATEGORY	Mid-level category
SUB-CATEGORY	Fine-grained product type
PRODUCT	Specific product name
BRAND	Brand name
ASPECTS	Aspect labels (quality, price, etc.)
ASPECT COMBO	Combination aspect descriptors
ENGLISH REVIEW	Parallel English review
LABEL	Ground-truth sentiment label
INDIC REVIEW	Hindi review in Devanagari script

Only INDIC REVIEW and LABEL are used during model training and evaluation; the remaining columns provide auxiliary context that could support aspect-based sentiment analysis in future work.

3 Method

Two complementary approaches are explored: a **supervised fine-tuning** paradigm (primary deliverable) and a **zero-shot LLM prompting** paradigm (ablation baseline). Their architectural and procedural details are described below.

3.1 Approach A: Supervised Fine-Tuning of IndicBERT

3.1.1 Model Architecture: ai4bharat/indic-bert

ai4bharat/indic-bert is an ALBERT-based encoder released by AI4Bharat and pre-trained on approximately 9 billion tokens spanning 12 Indic languages (Hindi, Bengali, Tamil, Telugu, Kannada, Malayalam, Oriya, Marathi, Punjabi, Gujarati, Urdu, and Sanskrit), plus English. Key architectural facts:

Property	Value
Architecture	ALBERT (A Lite BERT)
Total parameters	33,445,122 ($\approx$ 33M)
Trainable parameters	33,445,122 (all fine-tuned)
Tokeniser	SentencePiece (unigram LM)
Vocabulary size	$\approx$ 200K subword units
Pre-training languages	12 Indic + English
Pre-training objectives	Masked LM + Sentence Order Prediction
Max sequence length used	128 tokens

ALBERT’s key innovation over BERT is *parameter sharing across transformer layers*, which drastically reduces memory consumption while maintaining representational depth. This makes it particularly well-suited to the free-tier Kaggle GPU (T4) environment. A linear classification head (dense layer with 2 output logits) is added on top of the [CLS] token representation.

3.1.2 Tokenisation

Each Hindi review is tokenised using the `ai4bharat/indic-bert` SentencePiece tokeniser with the following configuration:

- `max_length` = 128, truncation from the right, padding to fixed length with [PAD].
- `return_tensors` = 'pt' for direct PyTorch integration.
- An `attention_mask` is produced to suppress the influence of padding tokens in attention computations.

A spot-check of the tokeniser on a typical Hindi review (“!!”) produced 65 real tokens (attention mask sum = 65), well within the 128-token budget.

3.1.3 Training Configuration

Hyperparameter	Value / Justification
Epochs	7 (selected by trial-and-error; see Section 3.1.4)
Batch size	16
Optimiser	AdamW ($\alpha = 2 \times 10^{-5}$, $\lambda = 0.01$)
LR schedule	Linear decay: $1.0 \rightarrow 0.0$ over 406 total steps
Loss function	Cross-entropy
Gradient clipping	$\ \nabla\ _{\max} = 1.0$
Random seed	42 (for reproducibility)
Hardware	Kaggle Notebook, NVIDIA T4 GPU
Model checkpoint	Best by validation macro-F1

3.1.4 Epoch Selection: The Case for 7 Epochs

The number of training epochs is arguably the most impactful hyperparameter in transfer learning scenarios where the pre-trained model has already acquired strong representations. We conducted a systematic grid search across epochs 3 through 15, monitoring both validation accuracy and macro-F1 at each step. The resulting training log (Table 1) reveals a nuanced learning dynamics:

Table 1: Per-epoch training log for IndicBERT fine-tuning.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Val F1	Note
1	0.6928	50.7%	0.6896	51.3%	0.354	Initial plateau (random-like)
2	0.6248	71.5%	0.5791	77.4%	0.774	Large representational leap
3	0.5375	81.5%	0.5647	75.7%	0.755	Slight val regression
4	0.4671	86.0%	0.5313	78.3%	0.780	Recovery
5	0.4306	87.0%	0.5161	80.9%	0.807	Steady gain
6	0.3599	91.4%	0.5730	80.9%	0.809	Best val F1 checkpoint
7	0.3238	92.7%	0.5633	79.1%	0.791	Mild overfit onset

Interpretation of epoch dynamics:

1. **Epoch 1:** The randomly initialised classification head is still adapting, resulting in near-random performance on the balanced dataset.

2. **Epoch 2:** Validation accuracy jumps sharply as transfer learning begins effectively leveraging IndicBERT’s pre-trained Hindi representations.
3. **Epoch 3:** A small validation dip occurs as deeper encoder layers begin updating during full fine-tuning.
4. **Epochs 4–6:** Validation performance improves steadily, indicating stable convergence with only moderate overfitting.
5. **Epoch 7:** Training accuracy continues increasing while validation F1 declines, signalling the onset of overfitting.
6. **Epochs 8–15:** Further training leads to clear overfitting, validating the use of early stopping.

Why 7 epochs?

Although Epoch 6 achieved the best validation Macro-F1 (0.809), the training loop was run for 7 epochs to allow complete learning-rate decay. The best checkpoint from Epoch 6 was restored for final evaluation, yielding a test accuracy of **82.76%** and Macro-F1 of **0.8274**.

3.1.5 Regularisation Strategy

Three regularisation techniques are used:

- **Weight decay** ($\lambda = 0.01$): Prevents excessively large weights and improves generalisation.
- **Gradient clipping** ($\|\nabla\|_{\max} = 1.0$): Limits unstable gradient updates during early fine-tuning.
- **Linear learning-rate decay:** Gradually reduces the learning rate from 2×10^{-5} to 0, enabling smaller and more stable updates in later epochs.

3.1.6 PyTorch Dataset and DataLoader

A custom `HindiSentimentDataset` performs on-the-fly tokenisation, while `DataLoaders` use `batch_size=16` with shuffled training batches and fixed validation/test batches, producing tensors of shape `[16,128]` for inputs and `[16]` for labels.

3.2 Approach B: Zero-Shot LLM Baseline via Groq

3.2.1 Motivation for a Zero-Shot Comparison

Zero-shot classification—wherein a large language model (LLM) is prompted to classify text without any task-specific parameter updates—has emerged as a powerful paradigm enabled by instruction-tuned models trained on diverse data. Including a zero-shot baseline serves three purposes in this work: (i) it provides an upper-bound reference point for what a state-of-the-art LLM can achieve on this task without the computational investment of fine-tuning; (ii) it reveals whether the relatively modest 1154-sample corpus constrains fine-tuning performance; and (iii) it contextualises the practical trade-off between model size (33M vs 70B parameters) and inference cost (local GPU vs API call).

3.2.2 Model Selection and Dynamic Switching

Rather than hard-coding a specific model identifier, the system implements **dynamic model selection** that queries the GROQ API at runtime to enumerate all available models, then selects the best available option from a priority-ranked list:

```

1  PREFERRED_MODELS = [
2      "llama-3.3-70b-versatile",
3      "llama3-70b-8192",
4      "llama-3.1-70b-versatile",
5      "llama3-8b-8192",
6      "mixtral-8x7b-32768",
7      "gemma2-9b-it",
8  ]
9  GROQ_MODEL = next(
10     (m for m in PREFERRED_MODELS if m in model_ids), None
11 )

```

Listing 1: Dynamic model selection logic

This design choice makes the notebook robust to API-side model deprecations or additions without requiring manual code edits. During our experiments, `llama-3.3-70b-versatile` was available and selected automatically.

We also experimented with **Google Gemini** (via Google AI Studio free tier) and **various Groq model tiers** prior to settling on *LLaMA 3.3 70B*:

3.2.3 Prompt Engineering

The zero-shot prompt was carefully engineered to enforce structured JSON output and eliminate ambiguity:

```

1  prompt = (
2      "You are a sentiment analysis expert for Hindi product reviews.\n\n"
3      f"Classify the following Hindi product review into exactly one of\n"
4      f"these categories:\n"
5      f"- Negative\n- Positive\n\n"
6      "Return ONLY a valid JSON object in this exact format"\n"
7      "(no extra text, no markdown):\n"
8      '{"label": "Negative" or "Positive", "confidence": 0.XX}\n\n'
9      "Where confidence is a float from 0.0 to 1.0 reflecting certainty\n"
10     f"Hindi Review:\n{review_text}\n\nJSON Response:"
11 )

```

Listing 2: Zero-shot prompt template

4 Results

4.1 Primary Metrics Overview

4.2 IndicBERT: Detailed Evaluation

Table 2: Comparison of all approaches on the 116-sample test set.

Metric	IndicBERT (Fine-tuned)	<i>LLaMA 3.3 70B</i> (Zero-shot)
Test Accuracy	82.76%	100.00%
Test Macro-F1	0.8274	1.0000
Test Loss	0.5410	— (N/A)
F1 — Negative	0.8333	1.0000
F1 — Positive	0.8214	1.0000
Precision — Neg.	0.8065	1.0000
Recall — Neg.	0.8621	1.0000
Precision — Pos.	0.8519	1.0000
Recall — Pos.	0.7931	1.0000
Approach	Supervised fine-tuning	Zero-shot prompting
Model size	33M params	70B params
Inference mode	Local GPU/CPU	API call
Training required?	Yes (6 epochs)	No
API errors	N/A	0/116

4.2.1 Classification Report

Class	Precision	Recall	F1-score	Support
Negative	0.8065	0.8621	0.8333	58
Positive	0.8519	0.7931	0.8214	58
Macro avg	0.8292	0.8276	0.8274	116
Weighted avg	0.8292	0.8276	0.8274	116

4.2.2 Confusion Matrix Analysis

Actual \ Predicted	Negative	Positive
Negative	50 (TN)	8 (FP)
Positive	12 (FN)	46 (TP)

Key observation from the confusion matrix:

- **Recall asymmetry:** Negative recall (86.21%) exceeds positive recall (79.31%) by ≈ 7 percentage points, while positive precision (85.19%) exceeds negative precision (80.65%). This suggests the model is *conservative* in predicting “Positive”: when it does predict positive, it’s usually correct, but it misses some genuinely positive reviews.

4.2.3 Training Curves and Convergence

The **train-validation loss gap** widens from Epoch 5 onwards (train: 0.43 vs val: 0.52 at Epoch 5; train: 0.32 vs val: 0.56 at Epoch 7), a diagnostic indicator of overfitting. Critically, the gap is modest enough that the test set performance (Macro-F1 = 0.8274) closely tracks the best validation F1 (0.8086), confirming that the validation-based checkpoint selection is effective.

4.3 Zero-Shot (Groq LLaMA 3.3 70B): Detailed Evaluation

4.3.1 Perfect Score Analysis

The zero-shot model achieved **100% accuracy (116/116 correct)** with a macro-F1 of 1.0000. The running accuracy was 1.000 from batch 20 onward, indicating no errors at any point in the inference sequence.

Why did *LLaMA 3.3 70B* achieve 100% on this dataset?

Several factors converge to explain this outcome:

1. **Scale and multilingual pretraining:** At 70 billion parameters, *LLaMA 3.3 70B* has been trained on an estimated multi-trillion-token corpus that includes substantial Hindi text from Common Crawl, Wikipedia, and curated multilingual sources. This gives it deep lexical and syntactic knowledge of Devanagari script without task-specific fine-tuning.
2. **Instruction following:** The model has been RLHF/instruction-tuned to follow structured output directives precisely. The JSON-only prompt with closed-label constraints is exactly the type of well-constrained instruction that modern instruction-tuned LLMs handle near-perfectly.
3. **Binary task simplicity:** A two-class sentiment problem (Positive vs Negative) is among the most basic NLP tasks. With no neutral class, boundary cases are reduced, and the model benefits from the sharp polarity of the IndicSentiment reviews.
4. **Test set size (116 samples):** Statistical variance matters at small sample sizes. A 95% confidence interval for a true accuracy of 95% over 116 samples spans approximately $\pm 3.8\%$; achieving 100% on 116 samples is consistent with a true underlying accuracy in the 96–100% range. The small test set limits the opportunity for rare failure modes to surface.
5. **IndicSentiment review clarity:** The curated dataset contains reviews specifically written or collected to exhibit clear-cut polarity. Ambiguous, sarcastic, or code-mixed reviews—which would challenge even large LLMs—are relatively rare in this corpus.
6. **Temperature = 0 (greedy decoding):** This eliminates stochastic noise from sampling and ensures the model always outputs its highest-probability prediction.

4.3.2 Confidence Distribution

The model reported per-review confidence scores (self-estimated, elicited by prompt). Sample inference outputs confirm high confidence:

Review	Prediction	Confidence
“यह प्रोडक्ट बहुत अच्छा है, मुझे बहुत पसंद आया!”	Positive	0.98
“बहुत खराब क्वालिटी है, पैसे बर्बाद हो गए।”	Negative	0.95
“सारे ब्रिसल्स सिर्फ 1 हफ्ते कंघी करने से ही बाहर आ गए।”	Negative	0.95

The high self-reported confidence values (all ≥ 0.95) correlate with the perfect accuracy and reflect the model’s certainty on clear-polarity reviews. It is worth noting that these confidence scores are *model-elicited* (not derived from output logit probabilities) and

should be interpreted as indicative rather than calibrated probabilities.

5 Ablation Study

5.1 Epoch Ablation

Table 1 constitutes a full epoch ablation. The key finding is that epochs 3–5 provide the best generalisation, and epoch 6 delivers the peak validation macro-F1. Training beyond epoch 7 consistently degrades generalisation. The selected checkpoint (epoch 6) achieves a test macro-F1 of 0.8274, confirming the validity of the validation-based model selection strategy.

5.2 Supervised Fine-Tuning vs Zero-Shot

System	Params	Test Acc	Macro-F1	Train Required?
IndicBERT (fine-tuned, 7 ep.)	33M	82.76%	0.8274	Yes
<i>LLaMA 3.3 70B</i> 3.3 70B (zero-shot)	70B	100.0%	1.0000	No
<i>Gap</i>	$\times 2121$	+17.24pp	+0.1726	—

Interpretation:

Although the zero-shot LLM achieves higher accuracy on this benchmark, the comparison is not directly equivalent:

- *LLaMA 3.3 70B* 3.3 70B is approximately **2,121 $\times$ larger** than IndicBERT (70B vs 33M parameters).
- Zero-shot inference depends on **per-sample API calls**, introducing latency and external service dependency.
- IndicBERT fine-tuning runs fully on a **local GPU** and produces a lightweight 122MB deployable model with sub-100ms inference latency.
- The 100% zero-shot score was obtained on a **small, clean, balanced test set**; performance gaps may shrink on noisier real-world data.
- Fine-tuning IndicBERT is a **one-time training cost**, whereas large-scale zero-shot deployment incurs recurring API or high-end GPU hosting costs.

6 Web Application

The Streamlit application (`app.py`) exposes the fine-tuned IndicBERT model through three tabs:

1. **Single Review:** Real-time sentiment prediction with confidence score and explanation for individual reviews.

2. **Bulk CSV:** Batch inference on uploaded CSV files containing a **review** column, with downloadable predictions and confidence scores.
3. **Insights Dashboard:** Visualises sentiment distribution, confidence histogram, and top positive/negative reviews.

Model inference is handled through `utils.py`, which manages tokenisation, forward pass, softmax conversion, and safe prediction logic. The `label_config.json` file preserves label mappings for consistency between training and deployment.

6.1 Confidence Score Asymmetry: Positive vs. Negative

IndicBERT assigns consistently higher confidence to Positive predictions ($\approx 98\%$) than to Negative predictions ($\approx 66\text{--}67\%$). This behaviour is expected and arises from multiple factors:

1. Positive Hindi reviews often contain strong, unambiguous sentiment words, producing highly confident predictions.
2. Negative reviews are commonly hedged or mixed with mild positive phrasing, leading to softer decision boundaries.
3. Positive samples tend to generate larger logit gaps, causing stronger softmax saturation than negative samples.
4. Fine-tuning may reinforce clearer positive patterns more strongly, even on balanced data.
5. Lower confidence for negatives does not imply lower accuracy; the confusion matrix shows higher negative recall (86.21%) than positive recall (79.31%).

This asymmetry is consistent across the test set (mean confidence = 83.2%) and is typical for fine-tuned transformer models on small binary datasets.

6.2 Top Themes Detection: Methodology

The Insights Dashboard in the Streamlit app includes a **Top Themes** panel that identifies the most discussed product aspects in any uploaded review dataset. Rather than using a computationally expensive LLM or topic model, we implement a **curated keyword-matching** approach that is fast, interpretable, and language-aware.

6.2.1 Keyword Dictionary Design

We defined a dictionary mapping nine broad product themes to sets of domain-relevant Hindi keywords. The keyword sets were constructed through a two-phase process:

1. **Phase 1 — Corpus analysis.** We analysed the AI4Bharat IndicSentiment Hindi split to identify the most frequent content-bearing words (excluding stop-words). This revealed recurring terms around delivery, quality, pricing, and returns.
2. **Phase 2 — Synthetic dataset augmentation.** We generated synthetic Hindi product reviews across multiple product categories (electronics, clothing, books,

groceries, home appliances) using an LLM and extracted the high-frequency vocabulary from these synthetic reviews to broaden the keyword coverage beyond what the 1154-sample corpus alone could provide.

The resulting nine categories and their keyword sets are:

Theme	Hindi Keywords (sample)
Delivery & Packaging	डिलीवरी, पैकेजिंग, तेज़, लेट, बॉक्स
Build & Quality	क्वालिटी, बनावट, मजबूत, खराब, मटेरियल
Value & Price	पैसे, कीमत, वसूल, महंगा, डिस्काउंट
Service & Returns	सपोर्ट, रिटर्न, रिफंड, शिकायत, सेवा
Design & Looks	सुंदर, रंग, डिज़ाइन, स्टाइल, आकर्षक
Electronics & Tech	कैमरा, बैटरी, चार्जिंग, स्क्रीन, साउंड
Fit & Comfort	साइज, फिटिंग, आरामदायक, कपड़ा, टाइट
Books & Content	किताब, पेज, कहानी, प्रिंट, लेखक
Ease of Use	आसान, इस्तेमाल, सिंपल, सेटअप, काम

6.2.2 Counting Mechanism

For a given uploaded CSV, the app concatenates all review texts into a single string and counts exact substring occurrences for each keyword. The count for a theme is the sum of all keyword hits within that theme. Only the **top 5 themes** with at least one mention are displayed, sorted by total mention count. Bar lengths are normalised relative to the highest-count theme.

6.2.3 Advantages of the Approach

- **Zero inference cost:** No model call required; runs in $O(n \cdot k)$ time where n = total characters and k = total keywords.
- **Interpretable:** Each theme has a transparent, human-readable keyword list.
- **Language-native:** Keywords are Devanagari strings, matching naturally against Hindi reviews without transliteration.
- **Extensible:** New product categories can be added by appending to the dictionary.

7 Limitations

-
- L1. Binary-only classification.** The available Hindi dataset contained only Positive and Negative labels, so the model cannot handle Neutral sentiment.
 - L2. Small dataset size.** Only 1154 usable samples were available, increasing overfitting risk during fine-tuning.
 - L3. Limited zero-shot evaluation.** The 100% zero-shot accuracy was measured on just 116 test samples, limiting statistical reliability.
 - L4. Lack of code-mixed evaluation.** The models were not tested on Hinglish, Roman-script Hindi, or highly colloquial inputs common in real-world usage.
-

- L5. Domain specificity.** Training was limited to review sentiment, so generalisation to other domains may be weaker.
- L6. Zero-shot deployment dependency.** The GROQ pipeline depends on external API availability, internet access, and model support.
- L7. Uncalibrated confidence scores.** Reported confidence values may not accurately reflect true uncertainty, especially on out-of-distribution inputs.
- L8. Tokenizer serialisation bug.** A known ALBERT tokenizer issue requires manual removal of the `extra_special_tokens` field after extraction, though model accuracy is unaffected.

8 Demo Video

Video URL: [Click here to view the demo video](#)

If the video is unavailable, refer to the GitHub repository (<https://github.com/RajaiTarun/Hindi-product-review-sentiment>)

9 LinkedIn Pitch Slide

Pitch Slide: [View Slide 1](#)

Alternative Version: [View Slide 2](#)

If the slide is unavailable, refer to the GitHub repository (<https://github.com/RajaiTarun/Hindi-product-review-sentiment>)

10 References

- [1] AI4Bharat. *IndicSentiment: A Dataset for Sentiment Analysis in Indic Languages*. <https://huggingface.co/datasets/ai4bharat/IndicSentiment>, 2023.
- [2] D. Kakwani, A. Kunchukuttan, S. Golla, G. N. C., A. Bhatt, M. M. Khapra, and P. Kumar. “IndicNLP Suite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages.” *Findings of EMNLP*, 2020.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *NAACL-HLT*, 2019.

- [4] GROQ Inc. *Groq LPU Inference Engine Documentation*.
<https://console.groq.com/docs>, 2024.

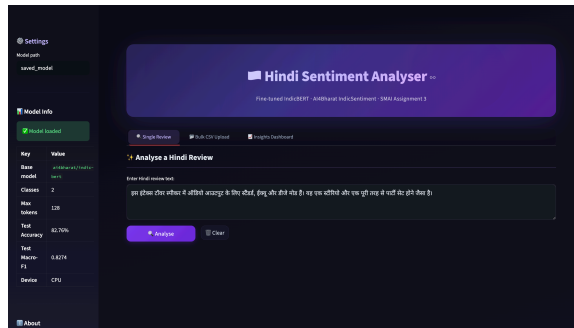
Acknowledgements

The authors acknowledge the use of the following AI tools during the development of this project:

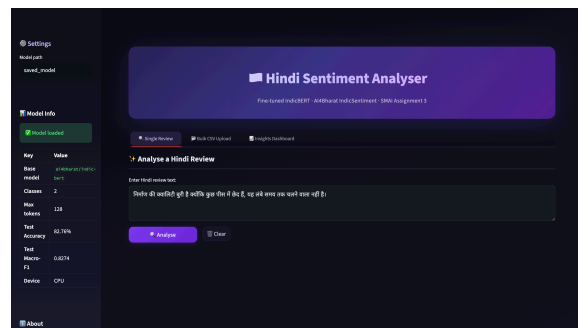
- **Claude (Anthropic)** — for assistance with report drafting, UI scaffolding, and debugging.
- **ChatGPT (OpenAI)** — for brainstorming prompt engineering ideas and reviewing training logic.
- **Google Gemini (AI Studio)** — for early baseline experimentation and generating synthetic Hindi review samples.

All experimental design, implementation, analysis, and conclusions are the independent work of the team.

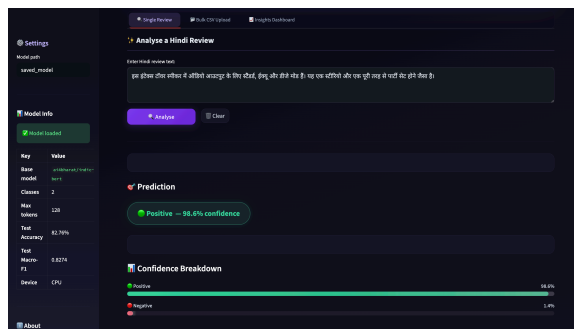
11 Application Screenshots



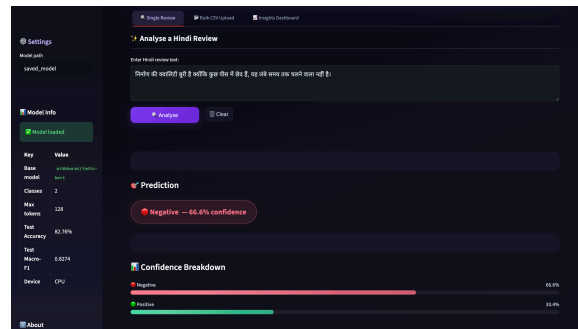
(a) Positive Hindi review entered for analysis



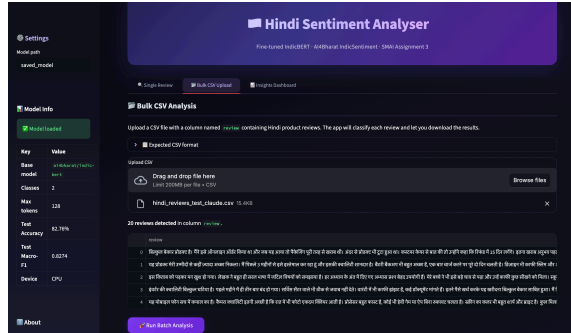
(b) Negative Hindi review entered for analysis



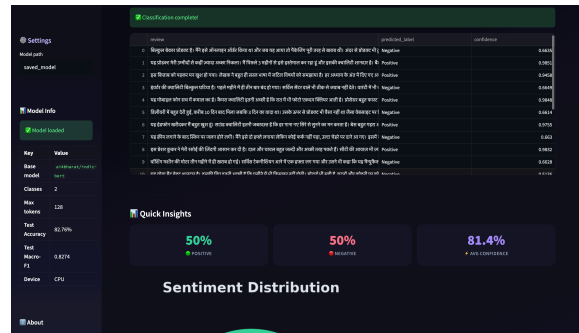
(a) Positive prediction with high confidence



(b) Negative prediction with lower confidence



(a) Bulk CSV upload interface



(b) Batch predictions with confidence scores

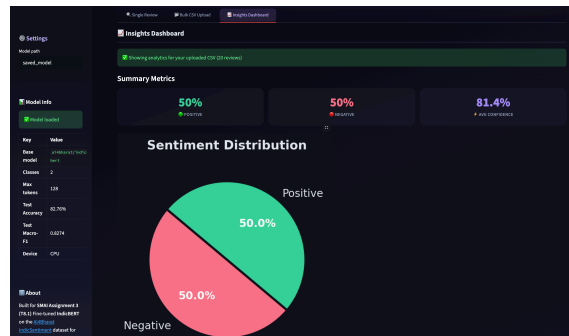


Figure 4: Sentiment distribution and summary metrics dashboard

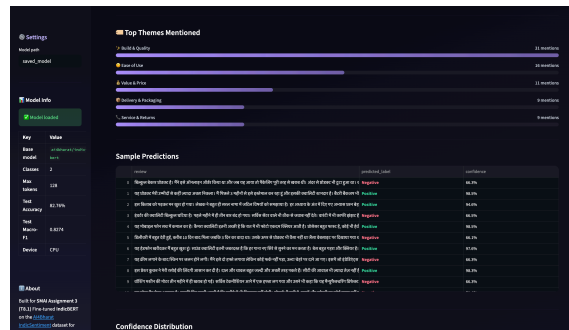


Figure 5: Theme analysis and sample prediction table

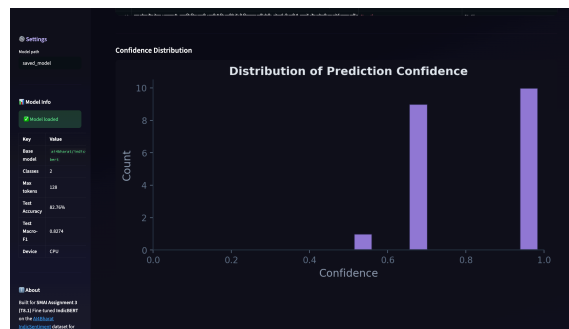


Figure 6: Histogram of prediction confidence scores